Lab Sheet — TP 1

Measuring Complexity Empirically

Corresponds to: Lectures 1–2 (see TD 1)

Session Objectives

- Translate pseudocode into working Python functions.
- Measure running time with the time module and plot it against input size.
- Estimate an unknown time complexity from measured data.

Exercise 1

Three functions below have different theoretical time complexities: $\Theta(n)$, $\Theta(n^2)$ and $\Theta(n \log n)$. Complete the starter code to measure and plot the running time of all three for growing n.

```
import time
import matplotlib.pyplot as plt

def linear_scan(n):
    total = 0
    for i in range(n):
        total += i
    return total

def quadratic_scan(n):
    total = 0
    for i in range(n):
        for j in range(n):
            total += 1
    return total

def sort_based(n):
    lst = list(range(n, 0, -1))
    return sorted(lst)

sizes = [100, 200, 400, 800, 1600]
times_linear, times_quad, times_sort = [], [], []

for n in sizes:
    start = time.perf_counter()
    linear_scan(n)
    times_linear.append(time.perf_counter() - start)
    # TODO: measure quadratic_scan(n) the same way, append to times_quad
    # TODO: measure sort_based(n) the same way, append to times_sort

plt.plot(sizes, times_linear, label='linear scan (Theta(n))')
# TODO: plot times_quad and times_sort with their own labels
plt.xlabel('n'); plt.ylabel('time (s)'); plt.legend(); plt.show()
```

Exercise 2

Two-Sum problem: given an array and a target value, decide whether some pair of elements sums to the target. Complete the optimised version below (it must run in Θ(n) time using a set), then measure and compare both implementations.

```
def two_sum_naive(arr, target):
    n = len(arr)
    for i in range(n):
        for j in range(i + 1, n):
            if arr[i] + arr[j] == target:
                return True
    return False

def two_sum_optimized(arr, target):
    # TODO: implement in Theta(n) time using a set
    pass

# Test cases (must pass before moving on)
assert two_sum_naive([2, 7, 11, 15], 9) == True
assert two_sum_naive([1, 2, 3], 100) == False
assert two_sum_optimized([2, 7, 11, 15], 9) == True
assert two_sum_optimized([1, 2, 3], 100) == False
print('All tests passed')
```

Exercise 3

Write a generic function estimate_exponent(f, sizes) that times f(n) for each n in sizes and estimates the exponent k such that the running time behaves like Θ(n^k), by fitting a line to (log n, log time) using numpy.polyfit (the slope of that line estimates k). Apply it to two_sum_naive and two_sum_optimized and check that the estimated exponents are close to 2 and 1 respectively.